

17. Une boucle qui dure plusieurs images

Un « for » se fait d'un coup ; pour qu'on VOIE quelque chose se construire, la boucle du jeu doit porter le compte.

1. Ce qu'il faut comprendre

C'est la leçon la plus importante du chapitre pour un jeu. Un `for` ne rend pas la main avant son dernier tour : tant qu'il tourne, rien d'autre ne se passe. Le `for` du haut pose ses 20 # écran éteint, **avant** que le jeu commence : c'est le bon moment pour tout faire d'un coup.

La ligne du bas se construit **une case toutes les 4 images**. Il n'y a **pas de** `for` : c'est la boucle du jeu elle-même qui fait les tours. La variable `pose` — déclarée **avant** la boucle, voir le chapitre 1 — retient **où l'on en est** d'une image à l'autre.

Toute animation d'un jeu est faite ainsi : une porte qui s'ouvre, une barre de vie qui descend, un texte qui s'écrit lettre par lettre. **On ne met jamais un long** `for` **qui attend au milieu du jeu** : pendant qu'il tourne, les touches ne sont plus lues, les ennemis ne bougent plus — le jeu est gelé. Même un `for` court à écrire se paie en images de jeu gelé s'il attend.

À retenir : un `for` sert à **tout faire maintenant** (dessiner un décor écran éteint, parcourir un tableau, calculer). Une **variable + la boucle du jeu** sert à **faire un peu à chaque image**.

Et même sans écrire à l'écran, un `for` trop long dans la boucle du jeu coûte cher : s'il ne tient plus dans 1/60 de seconde, le jeu **ralentit**. La leçon 76 du tutoriel mesure ce coût.

À toi : écris « BONJOUR » lettre par lettre, une toutes les 10 images, avec la même méthode.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **578 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         137 o ← démarrage - ta fonction
VBlank         8 o  ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← ecran() - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte - attend de pouvoir écrire une case
AdresseCase    10 o ← texte() - trouve l'adresse d'une case de l'écran
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles   76 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         192 o ← CopierTuiles - 12 lettres sur 43
"D UN COUP:"   10 o ← texte() - un texte gravé
"#"            1 o  ← texte() - un texte gravé
"PETIT A PETIT:" 14 o ← texte() - un texte gravé
"#"            1 o  ← texte() - un texte gravé
Lettres gravées : ACDEINOPTU:#
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier,
```

DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 4	58 o sur place, dans main()	+ AdresseCase (10 o), EcrireTexte (10 o)
ecran()	× 2	21 o sur place, dans main()	+ AttendreVBlank (16 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : dessine une ligne de # d'un coup, puis une autre PETIT À PETIT,  
// une case toutes les 4 images, pendant que le jeu tourne.  
//  
// L'idée : un travail long (une animation) ne se fait pas dans une boucle for  
// qui bloque tout ; on en fait un petit morceau à chaque image.  
  
#include <ecran> // éteint ou rallume l'écran  
#include <texte> // écrit un texte à l'écran
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

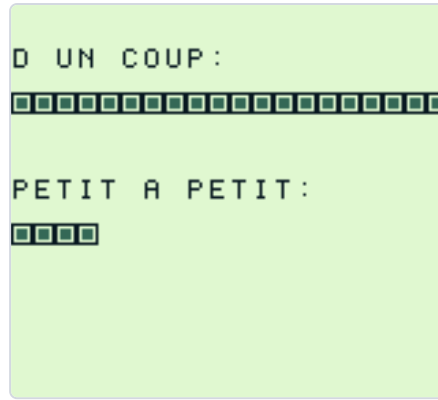
```
int main() { // Le jeu commence ici.  
    // D'un coup : écran éteint, avant que le jeu commence.  
    ekran(0); // Écran éteint.  
    texte(0, 2, "D UN COUP:");  
    for (uint8_t i = 0; i < 20; i++) { // 20 tours, i = 0 à 19 :  
        texte(i, 4, "#"); // les 20 # de la ligne 4.  
    }  
    texte(0, 8, "PETIT A PETIT:");  
    ekran(1); // On rallume : la ligne 4 apparaît entière.  
  
    // Petit à petit : une case toutes les 4 images.  
    uint8_t pose = 0; // Combien de # déjà posés sur la ligne 10 (0 à 20).  
    uint8_t attente = 0; // Les images attendues depuis le dernier #.  
  
    while (true) { // La boucle du jeu :  
        image(); // une image.  
  
        attente++; // Une image de plus.  
        if (attente == 4 && pose < 20) { // 4 images passées ET la ligne pas encore finie ?  
            texte(pose, 10, "#"); // pose un # à la colonne pose (0, puis 1, ...),  
            pose++; // un de plus de posé,  
            attente = 0; // et on recommence à attendre 4 images.  
        }  
    } // 20 cases × 4 images = 80 images ≈ 1,3 seconde.  
}
```

3. Ce qui se passe, étape par étape

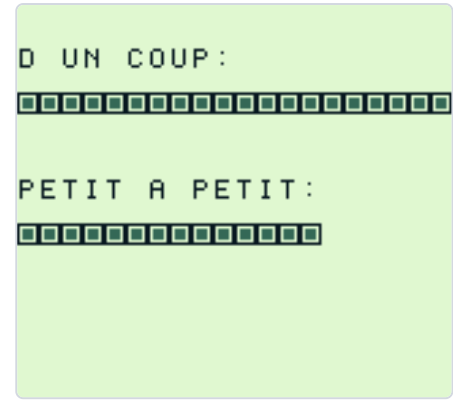
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



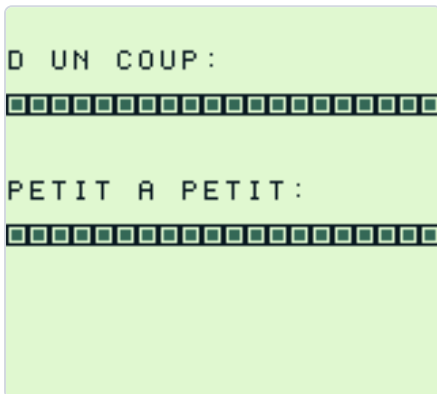
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

La ligne du haut est pleine d'emblée ; celle du bas se remplit case par case, en deux secondes environ.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le for a tout posé dès les premières images
- ✓ en bas, seulement quelques cases au début (4 cases)
- ✓ la ligne du bas grandit avec le temps (4 puis 9)
- ✓ elle finit par être pleine

← [16. do ... while : au moins une fois](#)

[sommaire](#)

[18. Un tableau : plusieurs cases sous un seul nom](#) →